Workload Management

DIRAC and DiracX as compute backends

Alexandre F. Boyer · CERN

12th DIRAC(X) Users' Workshop · FZU Prague · 13–16 October 2026



The end of two decades of homogeneity

> *Sites buy from the same market as everyone else: "Hardware technology evolution now is dominated by the needs of AI" (A. Sciabà, CERN IT, July 2026).*

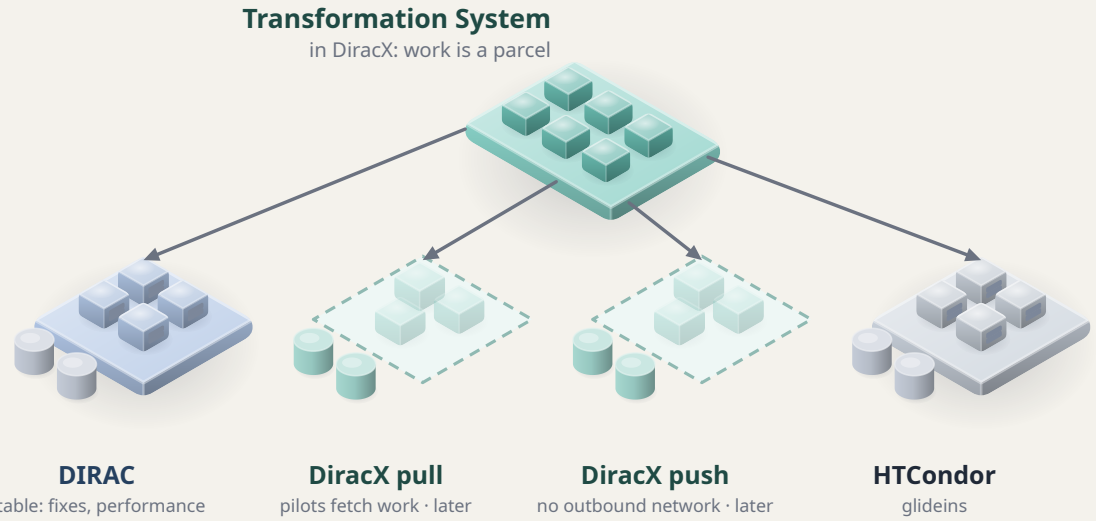
WHY IT CHANGES		WHAT SITES NOW OFFER		WHAT THE WMS MUST DO
AI expected productivity gains drive the market	→	GPUs, before they can be pledged or accounted	→	**Match on capabilities; run CPU and GPU work side by side**
Energy and hardware cost 2022 energy shock, climate targets, AI-driven prices	→	ARM next to x86: first WLCG ARM pledges in 2025	→	**One image name for every architecture; send each job to a node it can run on**
Identity X.509 gives way to the tokens industry uses	→	CEs that accept tokens; site accounting that still reads X.509 proxies	→	**Pilots carry a token *and* a proxy (DIRAC, stop 2); DiracX built on tokens**

In DIRAC: temporary solutions, to keep it stable. **In DiracX**: the long-term solutions, designed for this.

1 · Starting from the transformation system

What it changes for workload management

The transformation system sees compute backends



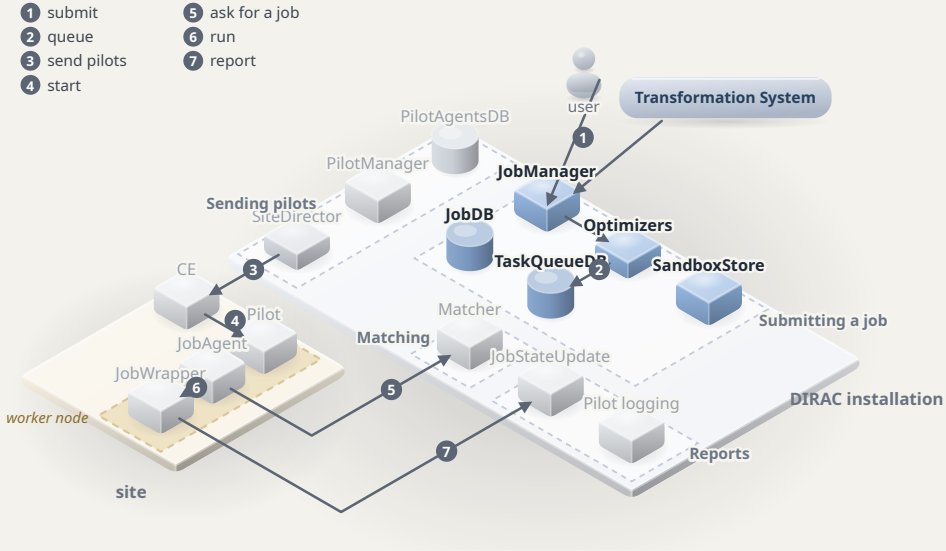
each backend keeps its own data and its own data model

The transformation system (Tuesday's talks) sends parcels to compute backends. **DIRAC is one of them**: it keeps its data, and gets fixes and performance only. DiracX is free of DIRAC's job model and **will get its own backends** (pull, push): not a priority, most communities rely on DIRAC or their own WMS for now (CMS talk).

2 · DIRAC: fixes and performance

Several fixes made it faster and more reliable, for v9 and above. A walk along the path.

Submitting a job



Jobs reach their task queue 1.6× faster, and only match where they should.

FASTER

- Jobs reach their task queue at ~190 jobs/s, not ~115; one queue instead of 64 duplicates during a burst (#8801, v9.1.20).
- Missing index on job names (#8463, v9.1.0, manual `ALTER TABLE`).

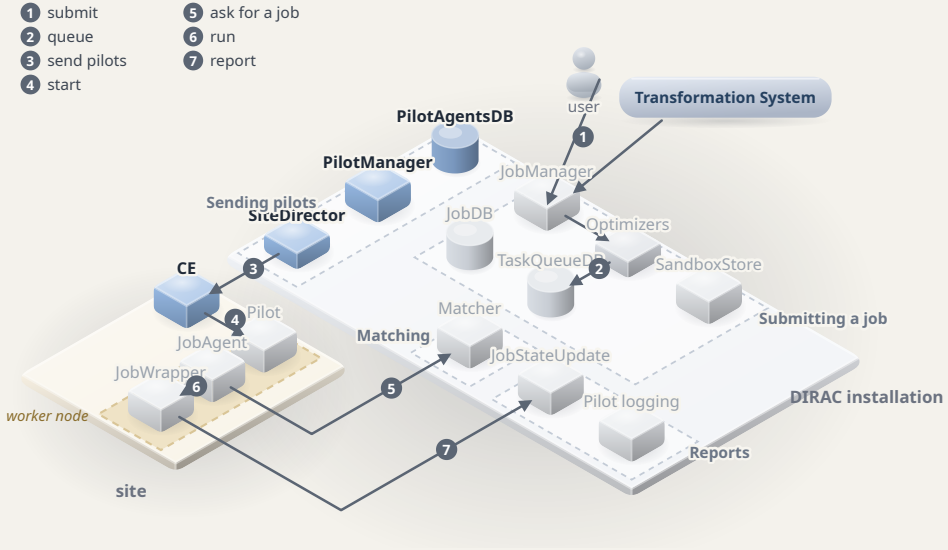
MORE RELIABLE

- A task queue could briefly lose its site restriction: a pilot at any site could then match jobs meant for other sites (#8801).

SAFER

- Sandbox store: a file name ending in `../` could write anywhere the service user can (GHSA-pwvr, fixed in v9.1.20, v9.0.26, v8.0.85).

Sending pilots



Pilots only go where they can run.

FASTER

- Indices for counting pilots per site and queue (#8463, manual `ALTER TABLE`).

MORE RELIABLE

- The SiteDirector no longer submits to banned CEs (#8340, v9.0.2).
- HPC: the SSH computing element, rewritten on Fabric, submits through a login node to the batch system (#7703, v9.1.3) `DUW11`.

SAFER

- PilotManager: SQL injection and missing access checks (GHSA-7xw9, fixed in v9.1.10, v9.0.22).
- Pilots with tokens, still with a proxy: next slide.

Tokens: why pilots still carry a proxy

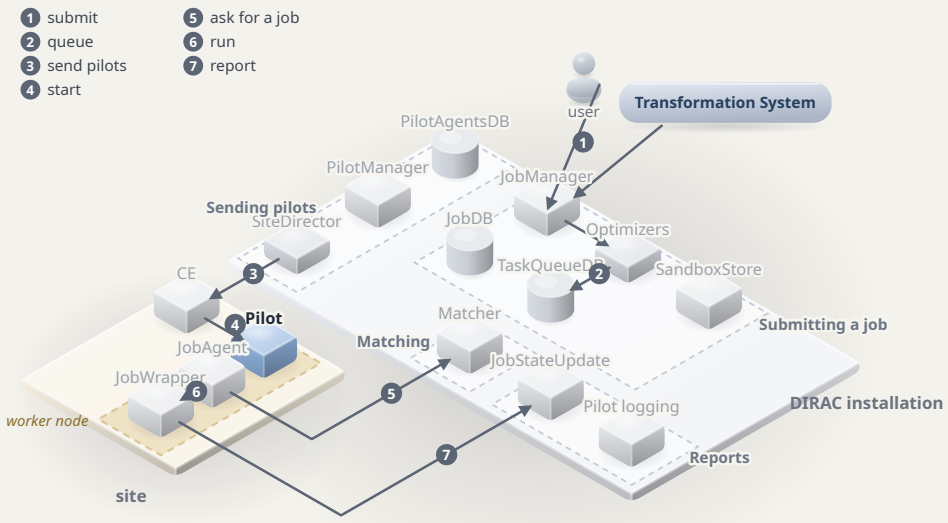
- Since v8.0 (#6803), the SiteDirector can submit pilots with a token, to CEs tagged `Token` or `Token:<VO>`.
- Sites still need the proxy that comes with the job: the accounting needs its VOMS information, and many ARC CEs need a proxy to run the job at all.
- So pilots must bring **both a token and a proxy**.

CE	with a token	
HTCondorCE	token **and** proxy	works
ARC (AREX)	token only	fails at many sites
ARC (AREX), `AlwaysIncludeProxy`	token and proxy	ARC6 uses the proxy only

On ARC, pilots are in practice submitted with a proxy, as before tokens (#7908, #7913).

No progress yet on token-only pilots: ARC7 can account from token claims, but some sites' accounting still relies on X.509 proxies. DiracX itself is built on tokens.

The pilot on the node



Python 3 only, one source of time left, a verified download.

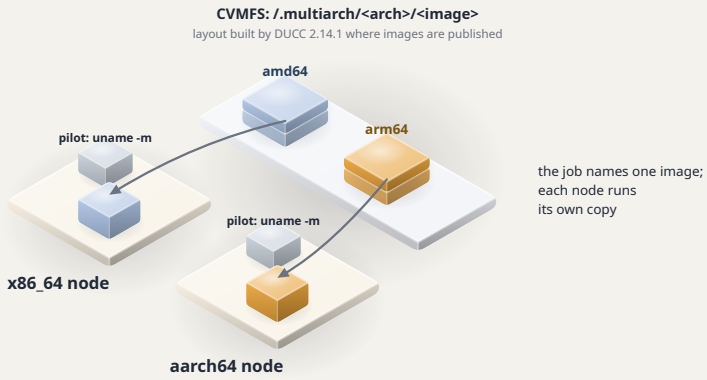
MORE RELIABLE

- The pilot is now Python 3 only: no more Python 2 code paths (Pilot #268) DUW11.
- The pilot reads the time left once and writes it to its configuration; jobs read it there, even in containers (#8479, v9.1.4).
- One image for every architecture (open PR): next slide.

SAFER

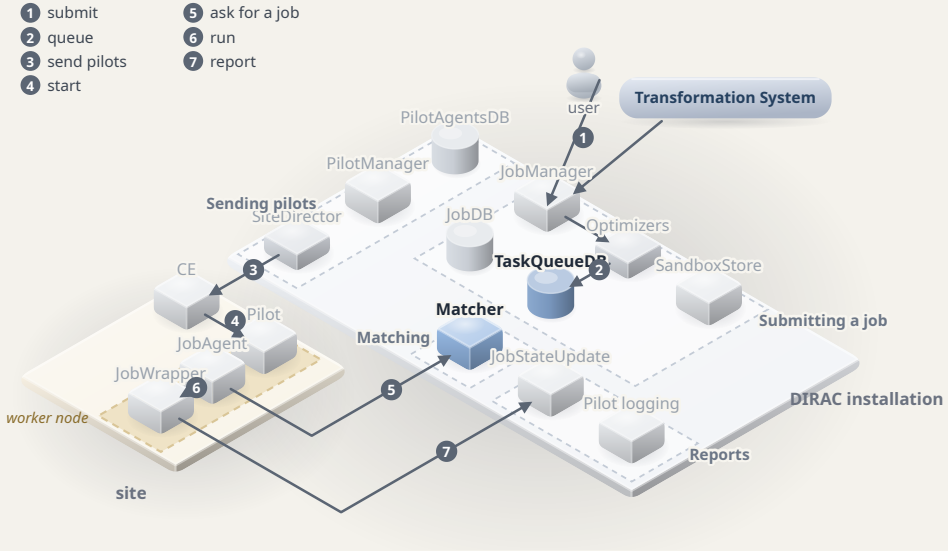
- The pilot code was fetched over HTTPS without checking the certificate, then run (GHSA-vg99, fixed in v9.1.10, v9.0.22).

One image for every architecture



- **CVMFS 2.14 supports multi-architecture images**: one image name, one copy per architecture, published by DUCC 2.14.1. Any CVMFS client reads them. We aim at relying on it.
- **In DIRAC**: a job names one container image, and the pilot runs the copy built for the node's architecture (x86_64 or aarch64). One of the last features added to DIRAC: #8456, open, to certify `DUW11`.

Matching

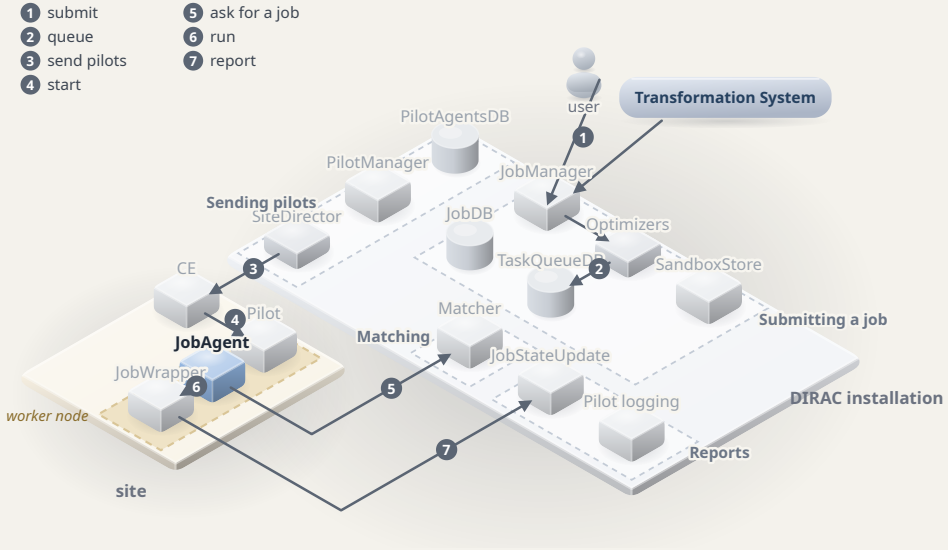


Matching up to ~17× faster.

FASTER

- Matching query: ~30 ms to ~3 ms (#8462, v9.1.0, manual `ALTER TABLE`).
- When 323 task queues could serve a pilot, finding them took 3.4 s; now 0.2 s (#8801, v9.1.20).

The JobAgent

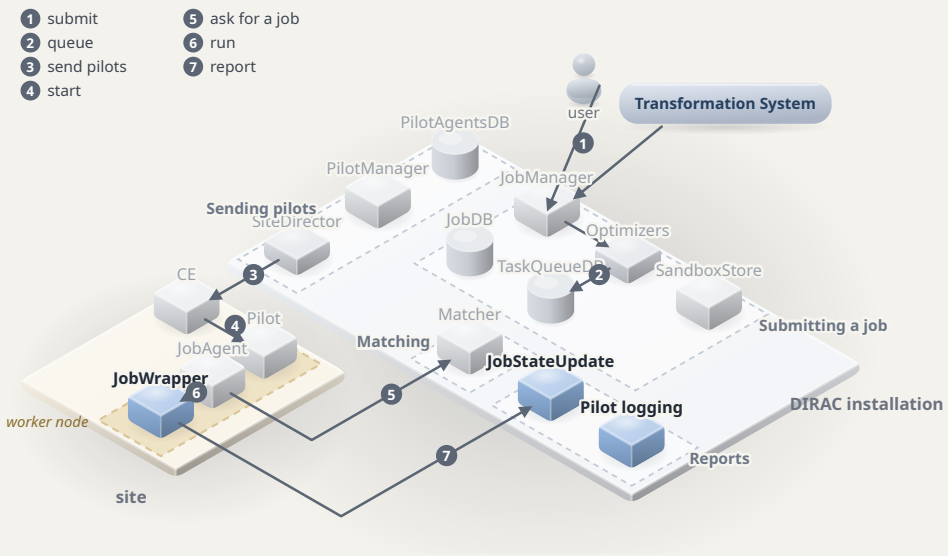


A job that restarts elsewhere is not failed by the node it left; each job keeps to its CPU and memory share.

MORE RELIABLE

- A job that cannot start (e.g. its input sandbox fails to download) is rescheduled, and may soon run at another site. Minutes later, the JobAgent on the first node saw the error and set the job to Failed, while it was running elsewhere. It now leaves rescheduled jobs alone (#8441, v9.0.17).
- Memory counts when matching and when filling a multi-core slot (#8366, #8414); cgroup v2 keeps each job within its CPU and memory share, where the site allows it (#8110) `DUW11`.

The JobWrapper and its reports



Reports DiracX accepts; nobody edits someone else's job.

MORE RELIABLE

- Jobs shorter than the first Watchdog cycle reported NaN: invalid JSON, so DiracX, which receives DIRAC's job reports, rejected the whole batch (#8750, v9.1.16).
- Time left, graceful stop, the reason of a kill: next slide. The JobWrapper in pull and push: the one after.

SAFER

- Job state updates: any user could change any job's owner or status (GHSA-j3xg).
- Pilot logging: a crafted file name could read any file on the server (GHSA-vm3r). Both fixed in v9.1.20, v9.0.26, v8.0.85.

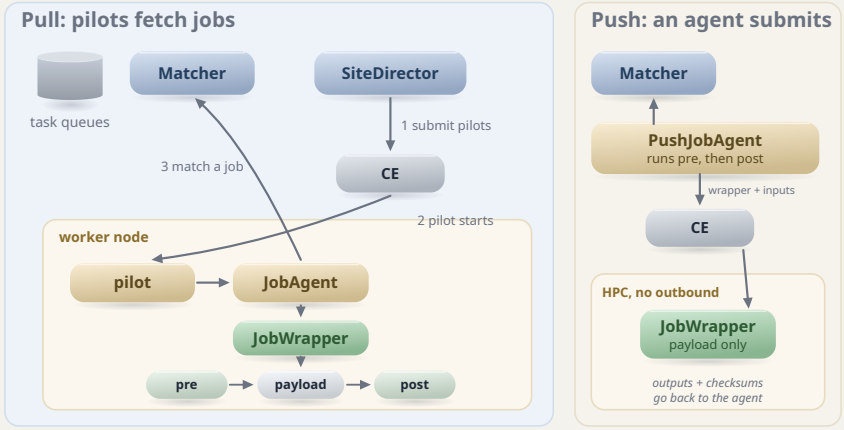
Time left in the slot: before and after



An example: an MC job sized for 183 events produced 150, was killed, and its output was never uploaded.

To use it: `StopMargin` and `StopSig*` in `/Systems/WorkloadManagement/JobWrapper`, or per job in the JDL; the graceful stop is off unless set. **v9.1.21 only.** Details: "Allocated time management" in the Administrator Guide. CPU work is normalised with DB12: next talk.

The JobWrapper in pull and push



- **Pull** (JobAgent, in the pilot): all three phases on the node, which needs outbound network.
- **Push** (PushJobAgent, on a DIRAC host): before and after run on the DIRAC host with the owner's proxy; the node only runs the payload, with DIRAC from CVMFS. Outputs come back with checksums.

Before: download the input sandbox, resolve the input data, the community's own pre-processing hook, prepare the command and environment.

Payload: runs under the Watchdog (CPU, memory, time left, stalled).

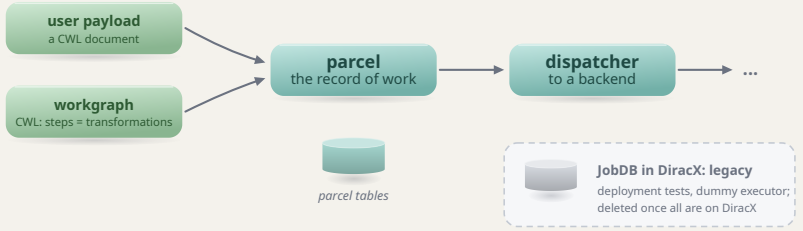
After: the community's own post-processing hook, upload the output sandbox, upload and register the output data, failover requests when an upload fails, accounting, final status.

Pre- and post-processing only exist since v9.0 (#7587), so communities still do these steps inside their workflows (e.g. uploading outputs). With CWL this should change: a CWL workflow must also run locally, so it holds no DIRAC-specific step; the job wrapper does them.

3 · DiracX: parcels, not jobs

And the proofs of concept around future DiracX compute backends

No jobs in DiracX

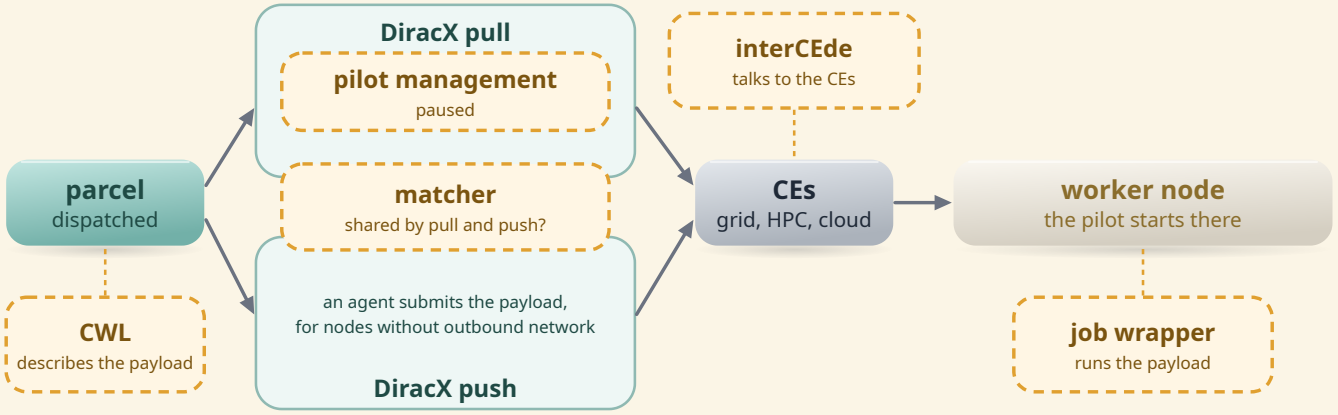


- A user payload is a parcel, like a bundle of transformation inputs. A failed parcel is final: resubmitting creates a new one.
- We do not port job monitoring or the job manager DUW11.

- The JobDB in DiracX is legacy: it receives job status updates from DIRAC, to **test a DiracX deployment**.
- It also feeds the **dummy job executor** (diracx#995): jobs go `Waiting → Done` without running anything, to test the transformation system. JobDB now, parcels later.
- **Once everyone is on DiracX, it is deleted.**

For you: in DIRAC, nothing changes until your community moves. In DiracX you will submit CWL: you can start writing it now, and we can help during the hackathon session.

Proofs of concept around the DiracX backends



CWL

ADR in draft; implemented with the transformation system

Pilot management

for DiracX pull; ported as is (diracx#570), paused: the data model needs redoing

Matcher

Python baseline and load tests; Redis next; maybe shared by pull and push

interCEde

ADRs in review; core proposed; integration tests

Job wrapper

requirements to gather: needs an owner

The same path for each: explore, write an ADR, then implement in DiracX, in small increments you can try.

CWL describes all the work

COMMON WORKFLOW LANGUAGE

```yaml
cwlVersion: v1.2
class: CommandLineTool
$namespaces: {dirac: https://diracgrid.org/cwl#}
hints:
  dirac:Job: {}        # a user payload (draft syntax)
baseCommand: [python, fit.py]
inputs:
  data: {type: File, inputBinding: {position: 1}}   # lfn:/... on the grid
outputs:
  plots: {type: File, outputBinding: {glob: "*.png"}}
```

WHY

One format for a user payload, a workgraph and each transformation. DiracX settings are `dirac:` hints: the same document runs on a laptop and at scale.

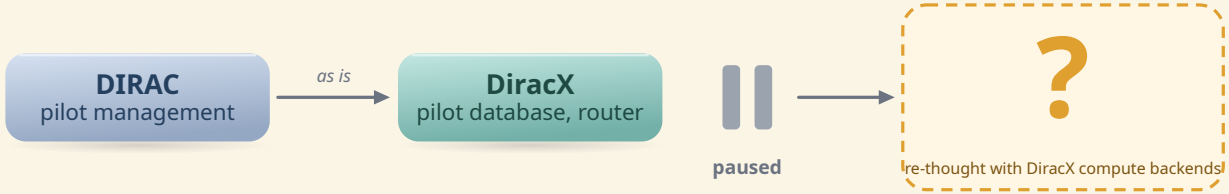
WHERE IT STANDS

DX-ADR-007, draft. `dirac-cwl` was a first proof of concept (with CTAO): only minor details may carry over. Upstream: cwltool#2302, #2166, schema_salad#1064 merged; cwl-utils#446 in review.

NEXT

Implemented with the transformation system, from this week's hackathon, where we can help you write your first CWL.

Pilot management: port paused



WHY

Pilots belong to the DiracX pull backend, which comes after the transformation system.

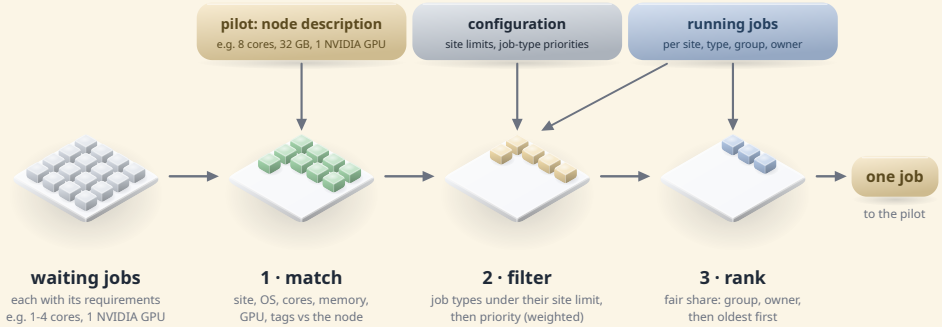
WHERE IT STANDS

Pilot database and router ported as is (diracx#570) DUW11. The data model could be much better, and changing it would mean large changes in DIRAC: paused.

NEXT

Re-thought with the DiracX compute backends. Meanwhile the DIRAC pilot keeps being patched.

A matcher for DiracX compute backends



one request at a time, in Python: every waiting job is checked

WHY

DIRAC's matcher will likely not be fast enough for what is coming: we aim for **1,000 requests per second**. It also cannot express alternatives: GPU A, or GPU B, or CPU C.

WHERE IT STANDS

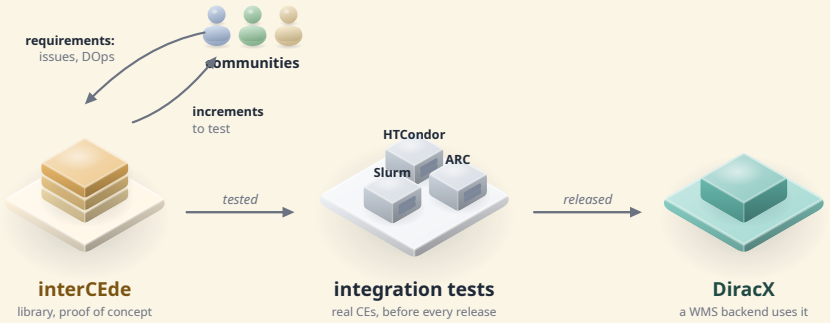
A deliberately simple Python matcher (POC), and load tests on LHCb-shaped data: 210 ms per match at 50k candidates.

NEXT

Redis, measured against this baseline, easy to rebuild (chaos tests with Redis off). Then an ADR, then DiracX.

Computing elements, explored from use cases

interCEde



WHY

Talking to CEs is needed by every WMS. Explored from the use cases DIRAC covers, and those it does not but should.

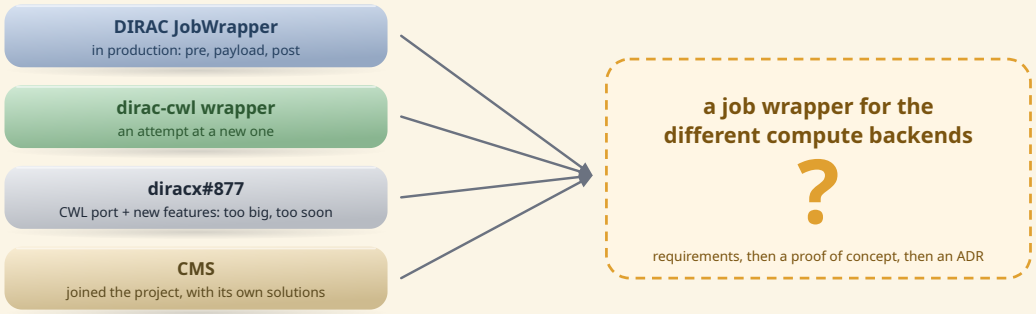
WHERE IT STANDS

ADRs under review; core contract proposed (interCEde#27). Its own repository: integration tests against real CEs, so a change cannot break production.

NEXT

Increments you can test. The ADRs are merged once a first implementation works.

The job wrapper: one for all of us



WHY

One JobWrapper in production. dirac-cwl: an attempt at a new one; diracx#877: its port with new features, too big, too soon. CMS joined, with interesting solutions.

WHERE IT STANDS

Requirements to gather: inputs, time left and graceful stop, containers, nodes without outbound network, reporting, failed uploads.

NEXT

Revisit what we have and what CMS brings, for the different compute backends: a proof of concept, then an ADR.

From you: someone, or a group of people (not necessarily the same for each step): requirements, proof of concept, ADR.

Thank you

Questions and comments welcome, this week and after.

Alexandre F. Boyer · alexandre.boyer@cern.ch

github.com/DIRACGrid · diracx.diracgrid.org



TAKE-AWAYS

- **DIRAC is one compute backend** of the transformation system. It stays stable: upgrade to **v9.1.21** (with its manual `ALTER TABLE` steps) for faster matching and insertion, the time-left fixes and the security patches.
- **DiracX works on parcels, not jobs.** Its own backends come later; interCEde and the matcher are explored now, with tests, and get an ADR after a first implementation.
- **Tokens**: pilots still carry a proxy, because some sites' accounting relies on it.

We need you: someone, or a group of people, for the job wrapper: requirements, proof of concept, ADR.